

Aim

To ingest real-time data (Twitter API / simulated sensor data) using **Apache Kafka**, process it using **Spark Structured Streaming**, and visualize insights using **Tableau / Power BI dashboards**.

Software & Hardware Requirements

Software

- Windows 10 / 11
- Java JDK 8 or 11
- Apache Kafka
- Apache Spark (with PySpark)
- Python 3.8+
- Tableau Public / Power BI Desktop

Hardware

- Minimum 8 GB RAM
 - 20 GB free disk space
-

Architecture Overview

Sensor / Twitter API

|
Kafka Producer
|

```
Kafka Topic
|
Spark Structured Streaming
|
Processed CSV / DB
|
Tableau / Power BI Dashboard
```

PART A: REAL-TIME DATA INGESTION USING KAFKA + SPARK STREAMING

Step 1: Install & Start Kafka

1.1 Start Zookeeper

```
bin\windows\zookeeper-server-start.bat config\zookeeper.properties
```

Output:

```
INFO binding to port 2181
INFO Started zookeeper server
```

1.2 Start Kafka Server

```
bin\windows\kafka-server-start.bat config\server.properties
```

Output:

```
INFO started (kafka.server.KafkaServer)
```

Step 2: Create Kafka Topic

```
bin\windows\kafka-topics.bat --create ^
```

```
--topic sensor-data ^
--bootstrap-server localhost:9092 ^
--replication-factor 1 ^
--partitions 1
```

Output:

```
Created topic sensor-data.
```

Step 3: Simulated Sensor Data Producer (Python)

3.1 Install Kafka Python Library

```
pip install kafka-python
```

3.2 Kafka Producer Code

```
from kafka import KafkaProducer
import json
import time
import random
from datetime import datetime

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

while True:
    data = {
        "sensor_id": random.randint(1, 5),
        "temperature": round(random.uniform(20, 40), 2),
        "humidity": round(random.uniform(30, 80), 2),
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    }
    producer.send("sensor-data", data)
```

```
print("Sent:", data)
time.sleep(2)
```

Output (Console):

```
Sent: {'sensor_id': 3, 'temperature': 31.5, 'humidity': 55.2,
'timestamp': '2025-12-22 14:30:01'}
```

Step 4: Spark Structured Streaming Consumer

4.1 Start PySpark

```
pyspark
```

4.2 Spark Streaming Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import *

spark = SparkSession.builder \
    .appName("KafkaSparkStreaming") \
    .getOrCreate()

schema = StructType([
    StructField("sensor_id", IntegerType()),
    StructField("temperature", FloatType()),
    StructField("humidity", FloatType()),
    StructField("timestamp", StringType())
])

df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "sensor-data") \
    .load()
```

```
json_df = df.select(from_json(col("value").cast("string"),
schema).alias("data")).select("data.*")

query = json_df.writeStream \
    .outputMode("append") \
    .format("csv") \
    .option("path", "output/sensor_data") \
    .option("checkpointLocation", "checkpoint") \
    .start()

query.awaitTermination()
```

4.3 Output Generated

Folder created:

```
output/sensor_data/
├── part-0000.csv
├── part-0001.csv
```

Sample CSV Output:

```
3,31.5,55.2,2025-12-22 14:30:01
1,28.7,62.9,2025-12-22 14:30:03
```

PART B: LIVE DATA VISUALIZATION USING TABLEAU / POWER BI

Step 5: Prepare Data for Dashboard

Merge CSV files (optional):

```
import pandas as pd
```

```
import glob

files = glob.glob("output/sensor_data/*.csv")
df = pd.concat([pd.read_csv(f, header=None) for f in files])
df.columns = ["Sensor ID", "Temperature", "Humidity", "Timestamp"]
df.to_csv("final_sensor_data.csv", index=False)
```

Step 6: Dashboard using Tableau

6.1 Open Tableau Public

- Click **Connect** → **Text File**
 - Select `final_sensor_data.csv`
-

6.2 Create Visualizations

Sheet 1: Temperature Over Time

- Columns → Timestamp
- Rows → Temperature
- Marks → Line

Sheet 2: Humidity by Sensor

- Columns → Sensor ID
- Rows → Humidity
- Marks → Bar

Sheet 3: Average Temperature

- Create Calculated Field:

`AVG([Temperature])`

6.3 Dashboard

- Combine all sheets
- Add filters for Sensor ID
- Enable auto-refresh

Dashboard Insights:

- Real-time temperature variations
 - Sensor-wise humidity comparison
 - Average temperature trends
-

Step 7: Dashboard using Power BI (Alternative)

Steps

1. Open **Power BI Desktop**
2. Get Data → Text/CSV
3. Load `final_sensor_data.csv`

Visuals

- Line chart → Temperature vs Timestamp
- Bar chart → Humidity vs Sensor ID

- Card → Average Temperature
-

Result

- ✓ Successfully ingested real-time sensor data using Kafka
 - ✓ Processed streaming data using Spark Structured Streaming
 - ✓ Stored processed Big Data into files
 - ✓ Developed interactive dashboards using Tableau / Power BI
-

Viva / Exam Questions

1. What is Kafka and why is it used in real-time systems?
 2. Difference between Spark Streaming and Structured Streaming
 3. Role of Zookeeper in Kafka
 4. How does Spark ensure fault tolerance in streaming?
 5. Advantages of Tableau / Power BI for Big Data analytics
-

Conclusion

This experiment demonstrates a complete **real-time Big Data pipeline**, starting from data ingestion to analytics visualization, enabling organizations to derive **real-time insights from streaming data**.